Soft Computing – Project.

Project 1: Breast Cancer Classification

[1] ✓ 13s

```
import numpy as np, pandas as pd
import matplotlib.pyplot as plt, seaborn as sns
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, classification_report, ConfusionMatrixDisplay

# 1. Data
data = load_breast_cancer(as_frame=True)
X, y = data.data, data.target          # 0 = malignant, 1 = benign
print("Shape:", X.shape)

# 2. EDA
fig, ax = plt.subplots(1, 2, figsize=(12, 4))
sns.countplot(x=y.map({0: 'malignant', 1: 'benign'}), ax=ax[0]); ax[0].set_title("Class balance")
X.corrwith(y).abs().sort_values(ascending=False).head(8).plot.barh(ax=ax[1])
ax[1].set_title("Top correlated features"); ax[1].invert_yaxis()
plt.tight_layout(); plt.show()

# 3. Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)
```

[1] ✓ 13s

```
# 4. Models
models = {
    "Logistic Regression": make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000)),
    "SVM (RBF)": make_pipeline(StandardScaler(), SVC(kernel='rbf')),
    "Neural Net (MLP)": make_pipeline(StandardScaler(),
        MLPClassifier(hidden_layer_sizes=(32, 16), max_iter=1000, random_state=42)),
}
results = {}
for name, m in models.items():
    m.fit(X_train, y_train)
    results[name] = {"Test Acc": accuracy_score(y_test, m.predict(X_test)),
                     "5-fold CV Acc": cross_val_score(m, X, y, cv=5).mean()}
res = pd.DataFrame(results).T.round(4)
print(res)
res.plot.bar(figsize=(7, 4), ylim=(0.9, 1.0)); plt.title("Model comparison"); plt.xticks(rotation=0); plt.show()

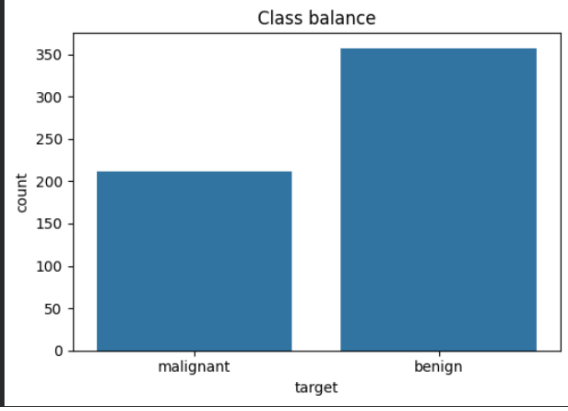
# 5. Best model analysis
best_name = res["5-fold CV Acc"].idxmax(); best = models[best_name]
pred = best.predict(X_test)
print("Best model:", best_name)
print(classification_report(y_test, pred, target_names=data.target_names))
ConfusionMatrixDisplay.from_predictions(y_test, pred, display_labels=data.target_names, cmap="Blues")
plt.title(f"Confusion Matrix - {best_name}"); plt.show()

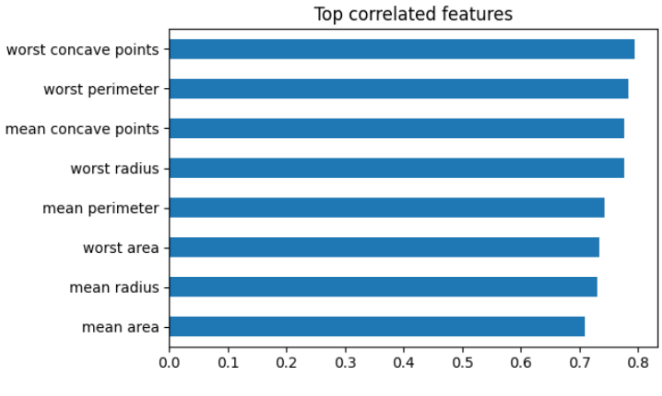
# 6. MLP loss curve
mlp = models["Neural Net (MLP)"].named_steps["mlpclassifier"]
plt.plot(mlp.loss_curve_); plt.xlabel("Iteration"); plt.ylabel("Loss"); plt.title("MLP training loss"); plt.show()
```

```
# 7. Single prediction
p = best.predict(X_test.iloc[[0]])[0]
print("Prediction:", data.target_names[p], "| Actual:", data.target_names[y_test.iloc[0]])
```

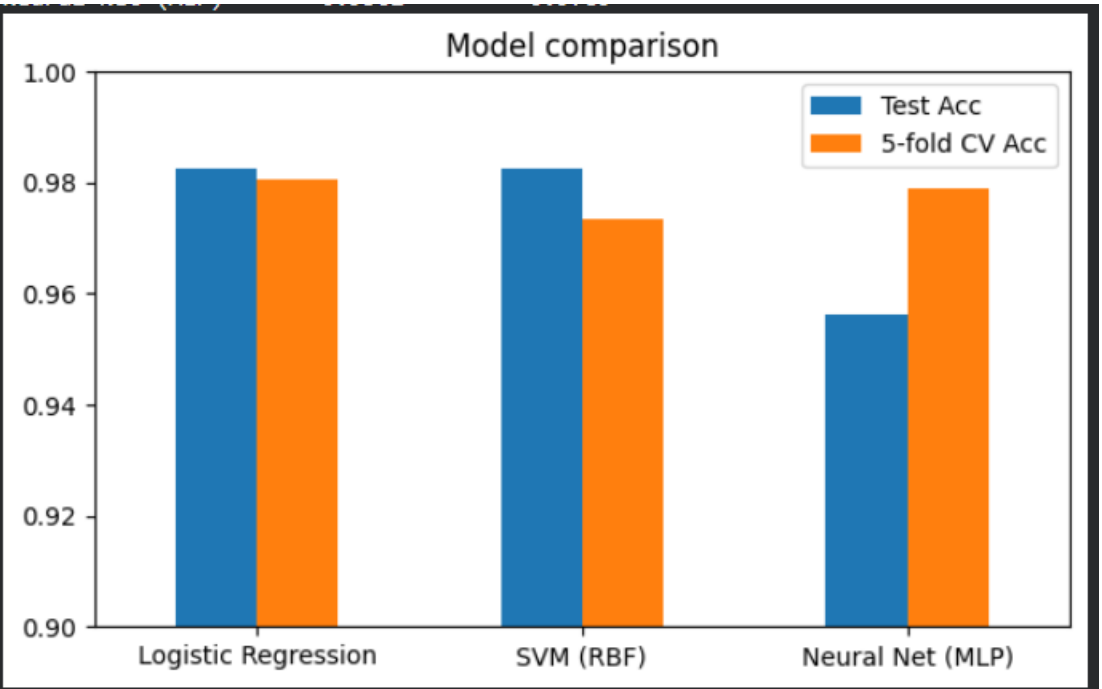
OUTPUT:

```
Shape: (569, 30)
```



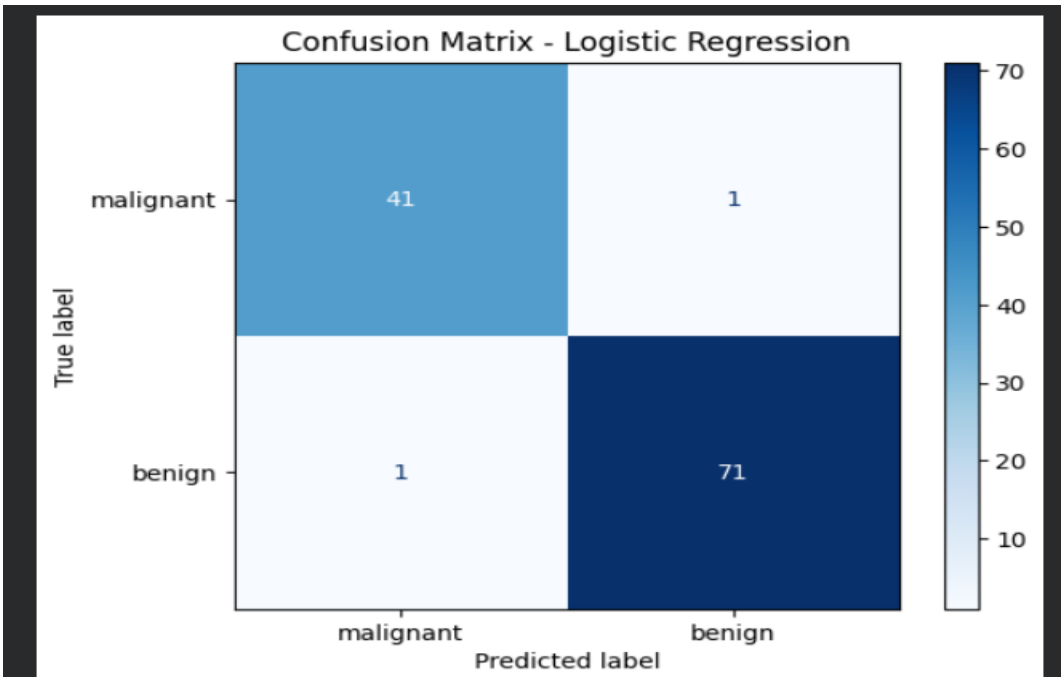


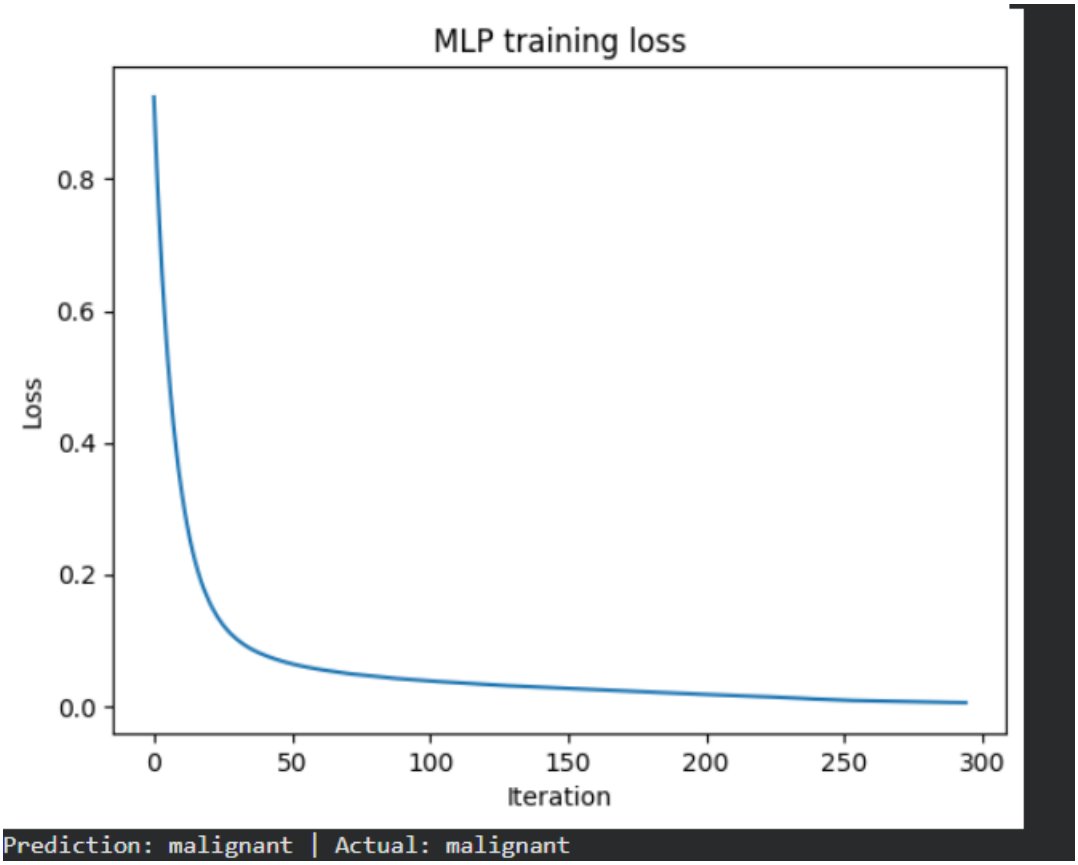
	Test Acc	5-fold CV Acc
Logistic Regression	0.9825	0.9807
SVM (RBF)	0.9825	0.9736
Neural Net (MLP)	0.9561	0.9789



Best model: Logistic Regression

	precision	recall	f1-score	support
malignant	0.98	0.98	0.98	42
benign	0.99	0.99	0.99	72
accuracy			0.98	114
macro avg	0.98	0.98	0.98	114
weighted avg	0.98	0.98	0.98	114






```
Prediction: malignant | Actual: malignant
```